SHERLOCK

# Security Review For Aztec Network

| | |
|---|---|
| Collaborative Audit Prepared For: | **Aztec Network** |
| Lead Security Expert(s): | **Koolex**<br>**oniki** |
| Date Audited: | **January 14 - January 28, 2026** |

# Introduction

This security review focused on the following two modules:

- The `cycle_group` module which provides implementations of in-circuit elliptic curve operations over the Grumpkin curve, the embedded curve for Barretenberg's BN254-based proving system. Grumpkin is a cofactor-1 curve defined over BN254's scalar field, making its base field operations native to the circuit.
- The memory module, i.e. ROM and RAM, in `barretenberg` which allow for a user to construct circuits with fixed-size memory tables, both with static and dynamic memory.

## Scope

Repository: AztecProtocol/aztec-packages

Audited Commit: c58cd76497e68c2236d875e7eac424f4b2d7bbd5

Final Commit: 6b858739756e0f56803e5d6a995f1308303a7f16

Files:

- barretenberg/cpp/src/barretenberg/circuit_checker/ultra_circuit_builder_-elliptic.test.cpp
- barretenberg/cpp/src/barretenberg/circuit_checker/ultra_circuit_builder_-memory.test.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/block_constraint.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/block_constraint.hpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/block_constraint.test.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/ec_operations.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/ec_operations.hpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/ec_operations.test.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/multi_scalar_mul.cpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/multi_scalar_mul.hpp
- barretenberg/cpp/src/barretenberg/dsl/acir_format/multi_scalar_mul.test.cpp
- barretenberg/cpp/src/barretenberg/relations/elliptic_relation.hpp
- barretenberg/cpp/src/barretenberg/relations/memory_relation.hpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/plookup_-tables/fixed_base/fixed_base.cpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/plookup_-tables/fixed_base/fixed_base.hpp

- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/plookup_-tables/fixed_base/fixed_base_params.hpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/plookup_-tables/fixed_base/fixed_base.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/rom_ram_logic.cpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/rom_ram_logic.hpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/ultra_circuit_-builder.cpp
- barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/ultra_circuit_-builder.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_group.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_group.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_group.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_scalar.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_scalar.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/cycle_scalar.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_lookup_-table.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_lookup_-table.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_lookup_-table.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_scalar_-slice.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_scalar_-slice.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/straus_scalar_-slice.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/group/test_utils.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/ram_table.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/ram_table.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/ram_table.test.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/rom_table.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/rom_table.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/rom_table.test.cpp

- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_-table.cpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_-table.hpp
- barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_-table.test.cpp
- barretenberg/cpp/src/barretenberg/ultra_honk/rom_ram.test.cpp

## Final Commit Hash

6b858739756e0f56803e5d6a995f1308303a7f16

## Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

## Issues Found

| High | Medium | Low/Info |
|---|---|---|
| 1 | 4 | 3 |

## Issues Not Fixed and Not Acknowledged

| High | Medium | Low/Info |
|---|---|---|
| 0 | 0 | 0 |

# Issue H-1: Unconstrained scalar limbs via constant point-at-infinity in batch multiplication breaks soundness [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/52

## Summary

In `cycle_group::batch_mul`, when a point is a constant point-at-infinity and the scalar is a witness, the function skips processing the scalar entirely via `continue`. This bypasses the range constraints that `cycle_scalar` depends on for soundness, leaving the scalar's `lo` and `hi` limbs unconstrained.

## Vulnerability Detail

The `cycle_scalar` constructor performs a partial validation (`validate_scalar_is_in_field`) that checks `lo + hi * 2128 < Grumpkin_modulus`, but this check **assumes** `lo < 2128` and `hi < 2126`. These range constraints are deferred to `batch_mul`, which applies them via `straus_scalar_slices` → `create_limbed_range_constraint`.

However, in `batch_mul` at line 990-995:

```
} else if (!scalar.is_constant() && point.is_constant()) {
    if (point.get_value().is_point_at_infinity()) {
        // ...
        continue;  // ← Scalar completely skipped!
    }
```

When the point is constant infinity, the `continue` bypasses scalar processing entirely, so range constraints are never applied.

## Impact

A malicious prover can create a `cycle_scalar` with out-of-range values (e.g., `lo = 2128`). If paired with a constant infinity point, that scalar's range constraints are never applied, breaking the soundness of `cycle_scalar` validation.

## Code Snippet

The `continue` skips scalar processing:

```
} else if (!scalar.is_constant() && point.is_constant()) {
    if (point.get_value().is_point_at_infinity()) {
```

```
        // oi mate, why are you creating a circuit that multiplies a known point at
        ↪  infinity?
#ifndef FUZZING_DISABLE_WARNINGS
        info("Warning: Performing batch mul with constant point at infinity!");
#endif
        continue;
    }
```

barretenberg/stdlib/primitives/group/cycle_group.cpp#L989-L996

The documented assumption that `batch_mul` violates:

```
* @warning The validation performed by this constructor is only sound if the
↪  resulting cycle_scalar is used in a
* scalar multiplication operation (batch_mul), which provides the necessary range
↪  constraints on lo and hi.
```

barretenberg/stdlib/primitives/group/cycle_scalar.cpp#L37-L39

## Proof of Concept

Two tests demonstrate the bug:

1. **SkipsRangeCheck**: Out-of-range scalar (`lo = 2128`) passes with constant infinity but fails with real point
2. **ZeroGates**: Gate count proves `batch_mul` adds nothing with constant infinity

```
TYPED_TEST(CycleGroupTest, TestBatchMulConstantInfinity_SkipsRangeCheck)
{
    STDLIB_TYPE_ALIASES;
    using field_ct = stdlib::field_t<Builder>;
    using fr = typename Builder::FF;

    fr bad_lo = fr(uint256_t(1) << 128);
    fr hi_val = fr(0);

    // Control: real point → should FAIL
    {
        auto builder = Builder();
        field_ct lo = field_ct::from_witness(&builder, bad_lo);
        field_ct hi = field_ct::from_witness(&builder, hi_val);
        cycle_scalar_ct scalar(lo, hi);
        cycle_group_ct point(TestFixture::generators[0]);
        (void)cycle_group_ct::batch_mul({ point }, { scalar });
        EXPECT_FALSE(CircuitChecker::check(builder));
    }

    // Bug: constant infinity → passes (range constraint skipped)
    {
```

```
        auto builder = Builder();
        field_ct lo = field_ct::from_witness(&builder, bad_lo);
        field_ct hi = field_ct::from_witness(&builder, hi_val);
        cycle_scalar_ct scalar(lo, hi);
        cycle_group_ct point = cycle_group_ct::constant_infinity(&builder);
        (void)cycle_group_ct::batch_mul({ point }, { scalar });
        EXPECT_TRUE(CircuitChecker::check(builder));
    }
}

TYPED_TEST(CycleGroupTest, TestBatchMulConstantInfinity_ZeroGates)
{
    STDLIB_TYPE_ALIASES;

    typename Group::Fr scalar_val = Group::Fr::random_element(&engine);

    size_t gates_scalar_only;
    {
        auto builder = Builder();
        cycle_scalar_ct scalar = cycle_scalar_ct::from_witness(&builder,
        ↪  scalar_val);
        (void)scalar;
        builder.finalize_circuit(false);
        gates_scalar_only = builder.get_num_finalized_gates();
    }

    size_t gates_with_infinity;
    {
        auto builder = Builder();
        cycle_scalar_ct scalar = cycle_scalar_ct::from_witness(&builder,
        ↪  scalar_val);
        cycle_group_ct point = cycle_group_ct::constant_infinity(&builder);
        (void)cycle_group_ct::batch_mul({ point }, { scalar });
        builder.finalize_circuit(false);
        gates_with_infinity = builder.get_num_finalized_gates();
    }

    size_t gates_with_real_point;
    {
        auto builder = Builder();
        cycle_scalar_ct scalar = cycle_scalar_ct::from_witness(&builder,
        ↪  scalar_val);
        cycle_group_ct point(TestFixture::generators[0]);
        (void)cycle_group_ct::batch_mul({ point }, { scalar });
        builder.finalize_circuit(false);
        gates_with_real_point = builder.get_num_finalized_gates();
    }

    EXPECT_EQ(gates_scalar_only, gates_with_infinity);
    EXPECT_GT(gates_with_real_point, gates_with_infinity);
```

```
}
```

**Run:**

```
cmake --preset default
cmake --build --preset default --target stdlib_primitives_tests
./build/bin/stdlib_primitives_tests --gtest_filter="*TestBatchMulConstantInfinity*"
```

**Output:**

```
[==========] Running 4 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 2 tests from CycleGroupTest/0, where TypeParam =
↪  bb::UltraCircuitBuilder_<bb::UltraExecutionTraceBlocks>
[ RUN      ] CycleGroupTest/0.TestBatchMulConstantInfinity_SkipsRangeCheck
(Experimental) WARNING: Builder failure when we have real witnesses! Ignore if
↪  writing vk. (mem: 18.23 MiB)
Failed Arithmetic relation at row idx = 1401 (mem: 26.11 MiB)
Failed at block idx = 2 (mem: 26.11 MiB)
Warning: Performing batch mul with constant point at infinity! (mem: 26.11 MiB)
[       OK ] CycleGroupTest/0.TestBatchMulConstantInfinity_SkipsRangeCheck (108 ms)
[ RUN      ] CycleGroupTest/0.TestBatchMulConstantInfinity_ZeroGates
Warning: Performing batch mul with constant point at infinity! (mem: 26.34 MiB)
[       OK ] CycleGroupTest/0.TestBatchMulConstantInfinity_ZeroGates (9 ms)
[----------] 2 tests from CycleGroupTest/0 (118 ms total)

[----------] 2 tests from CycleGroupTest/1, where TypeParam =
↪  bb::MegaCircuitBuilder_<bb::field<bb::Bn254FrParams> >
[ RUN      ] CycleGroupTest/1.TestBatchMulConstantInfinity_SkipsRangeCheck
(Experimental) WARNING: Builder failure when we have real witnesses! Ignore if
↪  writing vk. (mem: 26.34 MiB)
Failed Arithmetic relation at row idx = 1404 (mem: 26.73 MiB)
Failed at block idx = 4 (mem: 26.73 MiB)
Warning: Performing batch mul with constant point at infinity! (mem: 26.73 MiB)
[       OK ] CycleGroupTest/1.TestBatchMulConstantInfinity_SkipsRangeCheck (53 ms)
[ RUN      ] CycleGroupTest/1.TestBatchMulConstantInfinity_ZeroGates
Warning: Performing batch mul with constant point at infinity! (mem: 27.10 MiB)
[       OK ] CycleGroupTest/1.TestBatchMulConstantInfinity_ZeroGates (8 ms)
[----------] 2 tests from CycleGroupTest/1 (62 ms total)

[----------] Global test environment tear-down
[==========] 4 tests from 2 test suites ran. (220 ms total)
[  PASSED  ] 4 tests.
```

- **SkipsRangeCheck**: Control case fails ("Failed Arithmetic relation"), bug case passes – proving range constraint is skipped
- **ZeroGates**: Both pass – proving `batch_mul` adds zero gates with constant infinity point

## Tool Used

Manual Review

## Recommendation

When skipping a constant infinity point, still process the scalar through `straus_scalar_slices` to apply range constraints, or explicitly apply range constraints before the `continue`.

# Issue M-1: Degree-reduction optimization in doubling constraint breaks soundness for off-curve inputs [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/45

This issue has been acknowledged by the team but won't be fixed at this time.

## Summary

The point doubling constraint in `elliptic_relation.hpp` substitutes $x1^3$ with $y1^2 - curve_b$ for degree reduction. This substitution is only valid when points satisfy the curve equation $y^2 = x^3 + b$. Off-curve points can satisfy the constraint with incorrect results, breaking soundness.

## Vulnerability Detail

The doubling x-coordinate constraint should enforce:

```
(x3 + 2*x1) * 4*y1² = 9*x1
```

The implementation instead enforces:

```
(x3 + 2*x1) * 4*y1² = 9*x1*(y1² - b)
```

The comment in the code acknowledges this: *"N.B. we're using the equivalence x1^3 === y1^2 - curve_b to reduce degree by 1"*

This equivalence holds only for on-curve points. For off-curve point (2, 3) on Grumpkin (b = -17):

- $y1^2 = 9$ but $x1^3 + b = -9$, so the point is off-curve
- Correct constraint requires $x3 = 0$
- Buggy constraint accepts $x3 = 9$

## Impact

The doubling constraint produces incorrect results for off-curve points. Since the constraint's correctness depends on an invariant ($y^2 = x^3 + b$) that is not enforced at the circuit level, the proof system accepts invalid point doubling operations. This violates soundness.

## Code Snippet

```
// Contribution (3) point doubling, x-coordinate check
// (x3 + x1 + x1) (4*y1*y1) - 9 * x1 * x1 * x1 * x1 = 0
// N.B. we're using the equivalence x1^3 === y1^2 - curve_b to reduce degree by 1
const auto curve_b = get_curve_b();
auto x_pow_4_mul_3 = (Accumulator(y1_sqr_m - curve_b)) * x1_mul_3;
auto y1_sqr_mul_4_m = y1_sqr_m + y1_sqr_m;
y1_sqr_mul_4_m += y1_sqr_mul_4_m;
auto x1_pow_4_mul_9 = x_pow_4_mul_3 + x_pow_4_mul_3 + x_pow_4_mul_3;
auto x_double_identity = x3_plus_two_x1 * Accumulator(y1_sqr_mul_4_m) -
↪  x1_pow_4_mul_9;
std::get<0>(accumulators) += x_double_identity * q_elliptic_q_double_scaling;
```

aztec-packages/barretenberg/cpp/src/barretenberg/relations/elliptic_relation.hpp#L134

**Proof of Concept:**

Add to `barretenberg/cpp/src/barretenberg/circuit_checker/ultra_circuit_builder_elliptic.test.cpp`:

```
TEST_F(UltraCircuitBuilderElliptic, OffCurveDoublingBug)
{
    UltraCircuitBuilder builder;

    bb::fr x1(2);
    bb::fr y1(3);
    bb::fr curve_b = grumpkin::g1::curve_b;

    bb::fr y1_sq = y1.sqr();
    bb::fr rhs = bb::fr(9) * x1 * (y1_sq - curve_b);
    bb::fr x3 = rhs / (bb::fr(4) * y1_sq) - x1 - x1;

    bb::fr x1_sq = x1.sqr();
    bb::fr y3 = (bb::fr(3) * x1_sq * (x1 - x3)) / (y1 + y1) - y1;

    uint32_t x1_w = builder.add_variable(x1);
    uint32_t y1_w = builder.add_variable(y1);
    uint32_t x3_w = builder.add_variable(x3);
    uint32_t y3_w = builder.add_variable(y3);

    builder.create_ecc_dbl_gate({ x1_w, y1_w, x3_w, y3_w });

    // PASS
    EXPECT_TRUE(CircuitChecker::check(builder));
}
```

Run:

```
cmake --preset default
cmake --build --preset default --target circuit_checker_tests
./build/bin/circuit_checker_tests --gtest_filter="*OffCurveDoublingBug*"
```

Output:

```
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from UltraCircuitBuilderElliptic
[ RUN      ] UltraCircuitBuilderElliptic.OffCurveDoublingBug
[       OK ] UltraCircuitBuilderElliptic.OffCurveDoublingBug (5 ms)
[----------] 1 test from UltraCircuitBuilderElliptic (5 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (5 ms total)
[  PASSED  ] 1 test.
```

## Tool Used

Manual Review

## Recommendation

Ensure points are validated on-curve before entering the doubling operation.

# Issue M-2: `twin_rom_table` constant index access crashes due to uninitialized `entries` when no prior witness access [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/46

## Summary

`twin_rom_table::operator[](size_t)` does not call `initialize_table()` before accessing the `entries` vector. When constant index access occurs before any witness index access, the `entries` vector is empty, causing undefined behavior.

## Vulnerability Detail

The `twin_rom_table` class uses lazy initialization. The constructor populates `raw_entries` but leaves `entries` empty. The `entries` vector is only populated when `initialize_table()` is called.

The `operator[](field_pt)` correctly calls `initialize_table()` for non-constant indices. However, `operator[](size_t)` never calls it:

```
std::array<field_t<Builder>, 2> twin_rom_table<Builder>::operator[](const size_t
↪  index) const
{
    if (index >= length) {
        BB_ASSERT(context != nullptr);
        context->failure("twin_rom_table: ROM array access out of bounds");
    }
    return entries[index];  // entries is empty - no initialize_table() call
}
```

Additionally, `operator[](field_pt)` delegates to `operator[](size_t)` for constant indices *before* calling `initialize_table()`:

```
std::array<field_t<Builder>, 2> twin_rom_table<Builder>::operator[](const field_pt&
↪  index) const
{
    if (index.is_constant()) {
        return operator[](static_cast<size_t>(...));  // Delegates before
        ↪  initialize_table()
    }
    // ...
    initialize_table();  // Only reached for non-constant indices
}
```

Although `twin_rom_table` is used internally by `biggroup`, it always passes non-constant `field_ct` indices, so the vulnerable path is not triggered. However, as a public API in `bb::stdlib`, direct usage could trigger the segfault by performing constant index access before non-constant access.

The same bug exists in `rom_table::operator[](size_t)` – it also lacks an `initialize_table()` call. However, `rom_table` mitigates this through its `operator[](field_pt)` implementation, which calls `initialize_table()` before checking if the index is constant. This ensures the table is always initialized before delegating to `operator[](size_t)`.

In contrast, `twin_rom_table::operator[](field_pt)` checks for constant first and immediately delegates to `operator[](size_t)` – before ever reaching `initialize_table()`. This means `rom_table` is protected when accessed via `field_pt`, while `twin_rom_table` crashes in the same scenario.

## Impact

Undefined behavior (segmentation fault) when accessing the table with a constant index before any witness index access.

## Code Snippet

https://github.com/AztecProtocol/barretenberg/blob/master/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_table.cpp#L97-L106

aztec-packages/barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_table.cpp#L97-L106

**Proof of Concept:**

Add to `barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/twin_rom_table.test.cpp`:

```
/**
 * @brief PoC: operator[](size_t) accesses uninitialized entries vector
 *
 * BUG: operator[](size_t) does not call initialize_table(), so entries is empty.
 *
 * If you swap the order below (witness access first, then constant access),
 * the test passes because witness access calls initialize_table() which
 * populates the entries vector.
 */
TEST(TwinRomTable, ConstantAccessBeforeWitnessAccess)
{
    using Builder = UltraCircuitBuilder;
    using field_ct = stdlib::field_t<Builder>;
    using witness_ct = stdlib::witness_t<Builder>;
    using twin_rom_table_ct = stdlib::twin_rom_table<Builder>;
    using field_pair_ct = std::array<field_ct, 2>;
```

```
    Builder builder;

    std::vector<field_pair_ct> table_values;
    table_values.emplace_back(field_pair_ct{ witness_ct(&builder, 100),
    ↪  witness_ct(&builder, 200) });

    twin_rom_table_ct table(table_values);


    field_ct index = witness_ct(&builder, 0);

    // If you swap the following 2 lines, then table will be initialized and the
    ↪  test will pass
    auto constant_result = table[0];
    auto witness_result = table[index];

    EXPECT_EQ(witness_result[0].get_value(), 100);
    EXPECT_EQ(witness_result[1].get_value(), 200);
    EXPECT_EQ(constant_result[0].get_value(), 100);
    EXPECT_EQ(constant_result[1].get_value(), 200);
}
```

Run:

```
cmake --preset default
cmake --build --preset default --target stdlib_primitives_tests
./build/bin/stdlib_primitives_tests
↪  --gtest_filter="*ConstantAccessBeforeWitnessAccess*"
```

Output:

```
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from TwinRomTable
[ RUN      ] TwinRomTable.ConstantAccessBeforeWitnessAccess
Segmentation fault (core dumped)
```

## Tool Used

Manual Review

## Recommendation

Ensure the table is initialized before accessing entries in the constant index operator, consistent with the witness index operator behavior.

## Discussion

**koolexcrypto**

Or should return raw_entries[index] instead ? Not sure here, would like to know what do you think.

**notnotraju**

@koolexcrypto Thanks, my feeling is that `raw_entries[index]` is better for both `twin_rom` AND `row`, to avoid constructing another gate in the case of constant

# Issue M-3: `create_ecc_add_gate` missing `sign_coefficient` validation breaks elliptic curve relation assumption [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/48

## Summary

`create_ecc_add_gate` stores `sign_coefficient` directly into the `q_1` selector without validating that it equals +1 or -1. The elliptic curve relation assumes `q_sign² = 1` in its derivation. Invalid values (e.g., 0, 2) break the constraint math.

## Vulnerability Detail

The `create_ecc_add_gate` method validates wire indices but not `sign_coefficient`:

```
template <typename ExecutionTrace>
void UltraCircuitBuilder_<ExecutionTrace>::create_ecc_add_gate(const
↪  ecc_add_gate_<FF>& in)
{
    this->assert_valid_variables({ in.x1, in.x2, in.x3, in.y1, in.y2, in.y3 });
    // No validation of in.sign_coefficient

    auto& block = blocks.elliptic;

    // ...
    if (can_fuse_into_previous_gate) {
        block.q_1().set(block.size() - 1, in.sign_coefficient);  // Stored directly
        // ...
    } else {
        // ...
        block.q_1().emplace_back(in.sign_coefficient);  // Stored directly
        // ...
    }
}
```

The `sign_coefficient` becomes `q_sign` in the elliptic relation, which explicitly assumes `q_sign² = 1`:

```
// elliptic_relation.hpp:54
//   Constraint (via cancellation of denominator and assumption that q_sign^2 = 1):
//      (x3 + x1 + x2)(x2 - x1)^2 - (y2^2 - y1^2 + 2*q_sign*y2*y1) = 0
```

When `q_sign  {-1, +1}`, this equation no longer constrains valid elliptic curve operations.

## Impact

Circuit construction correctness issue. Invalid `sign_coefficient` values cause circuits to fail at constraint check time with a generic "Failed Elliptic relation" message, with no indication of the root cause.

## Code Snippet

aztec-packages/barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/ultra_circuit_builder.cpp#L383-L390

**Proof of Concept:**

Add to `barretenberg/cpp/src/barretenberg/stdlib_circuit_builders/ultra_circuit_builder_elliptic_test.cpp`:

```
/**
 * @brief PoC: sign_coefficient = 0 breaks the elliptic curve constraint
 *
 * BUG: create_ecc_add_gate does not validate sign_coefficient is +1 or -1.
 * The elliptic relation assumes q_sign^2 = 1 in its derivation.
 *
 * If you change sign_coefficient from 0 to 1, the test will fail because
 * the circuit becomes valid and CircuitChecker::check() returns true.
 */
TEST_F(UltraCircuitBuilderElliptic, InvalidSignCoefficientZero)
{
    UltraCircuitBuilder builder;
    auto points = create_add_points(1, 2, true);
    auto [x1, y1, x2, y2, x3, y3] = add_add_gate_variables(builder, points);

    builder.create_ecc_add_gate({ x1, y1, x2, y2, x3, y3, /*sign_coefficient=*/0 });

    EXPECT_FALSE(CircuitChecker::check(builder));
}
```

Run:

```
cmake --preset default
cmake --build --preset default --target ultra_circuit_builder_tests
./build/bin/ultra_circuit_builder_tests
↪  --gtest_filter="*InvalidSignCoefficientZero*"
```

Output with `sign_coefficient = 0` (constraint broken, test passes):

```
[==========] Running 1 test from 1 test suite.
[----------] 1 test from UltraCircuitBuilderElliptic
[ RUN      ] UltraCircuitBuilderElliptic.InvalidSignCoefficientZero
Failed Elliptic relation at row idx = 0 (mem: 7.91 MiB)
```

```
Failed at block idx = 4 (mem: 7.91 MiB)
[       OK ] UltraCircuitBuilderElliptic.InvalidSignCoefficientZero (12 ms)
[  PASSED  ] 1 test.
```

Output with `sign_coefficient = 1` (valid circuit, test fails):

```
[==========] Running 1 test from 1 test suite.
[----------] 1 test from UltraCircuitBuilderElliptic
[ RUN      ] UltraCircuitBuilderElliptic.InvalidSignCoefficientZero
Value of: CircuitChecker::check(builder)
  Actual: true
Expected: false
[  FAILED  ] UltraCircuitBuilderElliptic.InvalidSignCoefficientZero (5 ms)
[  FAILED  ] 1 test.
```

## Tool Used

Manual Review

## Recommendation

Add validation in `create_ecc_add_gate`:

```
ASSERT(in.sign_coefficient == 1 || in.sign_coefficient == -1);
```

# Issue M-4: Non-deterministic ROM Record ordering causes inconsistent circuit serialization [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/50

## Summary

The `RomRecord::operator<` comparator only compares by `index`, causing `std::sort` to produce non-deterministic ordering for records with equal indices across different compilers. This results in `export_circuit()` generating different serialized bytes for the same circuit on different platforms, breaking reproducibility for SMT verification tooling.

## Vulnerability Detail

When multiple ROM operations (init or read) target the same index, multiple `RomRecord` objects are created with identical `index` values but different `gate_index` values. The comparator at `rom_ram_logic.hpp:35` provides no tie-breaker:

```
bool operator<(const RomRecord& other) const { return index < other.index; }
```

In contrast, `RamRecord::operator<` at `rom_ram_logic.hpp:66-70` correctly includes a tie-breaker on `timestamp`:

```
bool operator<(const RamRecord& other) const
{
    bool index_test = (index) < (other.index);
    return index_test || (index == other.index && timestamp < other.timestamp);
}
```

## Non-determinism Mechanism

The C++ standard does not guarantee the relative ordering of equivalent elements after `std::sort`. Different standard library implementations (GCC's libstdc++, Clang's libc++, MSVC's STL) use different sorting algorithms that produce different orderings for elements that compare equal.

ROM records are sorted twice during the circuit export flow:

1. **`process_ROM_array()`** at `rom_ram_logic.cpp:200-204` sorts records during `finalize_circuit()`:

```
std::sort(rom_array.records.begin(), rom_array.records.end());
```

2. **`export_circuit()`** at `ultra_circuit_builder.cpp:2050-2063` sorts records again before serialization:

```
for (auto& rom_table : this->rom_ram_logic.rom_arrays) {
    std::sort(rom_table.records.begin(), rom_table.records.end());
    ...
}
```

Records with the same `index` end up in different orders depending on the compiler, causing their witness values (`index_witness`, `value_column1_witness`, `value_column2_witness`) to be serialized in different sequences.

## Proof of Concept

The following demonstrates that `std::sort` with an insufficient comparator produces different results across compilers, simulating the double-sort flow in the actual code:

```
#include <iostream>
#include <vector>
#include <algorithm>

struct RecordBroken {
    int index;
    int id;
    bool operator<(const RecordBroken& other) const {
        return index < other.index;
    }
};

struct RecordFixed {
    int index;
    int id;
    bool operator<(const RecordFixed& other) const {
        return index < other.index || (index == other.index && id < other.id);
    }
};

void print_index1(const std::vector<RecordBroken>& records, const char* label) {
    std::cout << label << ": ";
    for (const auto& r : records) {
        if (r.index == 1) std::cout << r.id << " ";
    }
    std::cout << std::endl;
}

void print_index1_fixed(const std::vector<RecordFixed>& records, const char* label)
↪ {
    std::cout << label << ": ";
    for (const auto& r : records) {
        if (r.index == 1) std::cout << r.id << " ";
```

```
    }
    std::cout << std::endl;
}

int main() {
    std::vector<RecordBroken> broken;
    std::vector<RecordFixed> fixed;

    for (int i = 0; i < 20; i++) {
        broken.push_back({i % 3, i});
        fixed.push_back({i % 3, i});
    }

    std::cout << "=== Without tie-breaker ===" << std::endl;
    print_index1(broken, "Before sort");

    std::sort(broken.begin(), broken.end());
    print_index1(broken, "After sort 1");

    std::sort(broken.begin(), broken.end());
    print_index1(broken, "After sort 2");

    std::cout << std::endl << "=== With tie-breaker ===" << std::endl;
    print_index1_fixed(fixed, "Before sort");

    std::sort(fixed.begin(), fixed.end());
    print_index1_fixed(fixed, "After sort 1");

    std::sort(fixed.begin(), fixed.end());
    print_index1_fixed(fixed, "After sort 2");

    return 0;
}
```

**GCC/Clang output:**

```
=== Without tie-breaker ===
Before sort: 1 4 7 10 13 16 19
After sort 1: 10 19 16 13 7 4 1
After sort 2: 13 1 4 7 16 19 10

=== With tie-breaker ===
Before sort: 1 4 7 10 13 16 19
After sort 1: 1 4 7 10 13 16 19
After sort 2: 1 4 7 10 13 16 19
```

**MSVC output:**

```
=== Without tie-breaker ===
```

```
Before sort: 1 4 7 10 13 16 19
After sort 1: 1 4 7 10 13 16 19
After sort 2: 1 4 7 10 13 16 19

=== With tie-breaker ===
Before sort: 1 4 7 10 13 16 19
After sort 1: 1 4 7 10 13 16 19
After sort 2: 1 4 7 10 13 16 19
```

**Analysis:**

- GCC/Clang's `std::sort` actively reorders equal elements on each sort
- MSVC's `std::sort` happens to preserve order (but this is not guaranteed by the standard)
- With the tie-breaker, both compilers produce identical deterministic results

*Note: Use Compiler Explorer (godbolt.org) to reproduce these results across different compilers.*

## Impact

**Serialization reproducibility broken**: Same circuit produces different `export_circuit()` bytes across platforms (GCC vs MSVC vs Clang). Breaks hash-based caching and direct binary comparison. Other impacts may exist where deterministic record ordering is assumed.

## Code Snippet

`rom_ram_logic.hpp:35`:

```
bool operator<(const RomRecord& other) const { return index < other.index; }
```

`rom_ram_logic.cpp:200-204`:

```
std::sort(rom_array.records.begin(), rom_array.records.end());
```

`ultra_circuit_builder.cpp:2050-2063`:

```
for (auto& rom_table : this->rom_ram_logic.rom_arrays) {
    std::sort(rom_table.records.begin(), rom_table.records.end());

    std::vector<std::vector<uint32_t>> table;
    table.reserve(rom_table.records.size());
    for (const auto& rom_entry : rom_table.records) {
        table.push_back({
            this->real_variable_index[rom_entry.index_witness],
            this->real_variable_index[rom_entry.value_column1_witness],
```

```
            this->real_variable_index[rom_entry.value_column2_witness],
        });
    }
    cir.rom_records.push_back(table);
    cir.rom_states.push_back(rom_table.state);
}
```

## Tool Used

Manual Review

## Recommendation

Add a tie-breaker to ensure deterministic ordering, consistent with `RamRecord::operator<`:

```
bool operator<(const RomRecord& other) const {
    return index < other.index || (index == other.index && gate_index <
    ↪  other.gate_index);
}
```

This fix ensures `std::sort` produces consistent ordering regardless of compiler or platform.

# Issue L-1: Default move constructor leaves moved-from table objects in inconsistent state [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/47

## Summary

The default move constructor in `rom_table`, `ram_table`, and `twin_rom_table` moves vectors (leaving them empty) but copies primitive members like `length` unchanged. This creates an inconsistent state where `size()` returns the original value but internal vectors are empty, causing a segmentation fault if the moved-from object is accessed.

## Vulnerability Detail

After moving a `rom_table`:

- `entries.size() = 0` (vector was moved)
- `length = original value` (primitive was copied)

The `operator[]` bounds check uses `length`, not `entries.size()`:

```
field_t rom_table::operator[](size_t index) {
    if (index >= length) {  // Passes because length is still > 0
        // error handling
    }
    return entries[index];  // UB: entries is empty
}
```

## Impact

Accessing a moved-from table object causes a segmentation fault. The bounds check passes but the subsequent vector access crashes due to the inconsistent state.

## Code Snippet

```
template <IsUltraOrMegaBuilder Builder> rom_table<Builder>::rom_table(rom_table&&
↪  other) = default;
```

barretenberg/stdlib/primitives/memory/rom_table.cpp#L113

```
template <IsUltraOrMegaBuilder Builder> ram_table<Builder>::ram_table(ram_table&&
↪  other) = default;
```

barretenberg/stdlib/primitives/memory/ram_table.cpp#L119

```
template <typename Builder>
↪  twin_rom_table<Builder>::twin_rom_table(twin_rom_table&& other) = default;
```

barretenberg/stdlib/primitives/memory/twin_rom_table.cpp#L91

**Proof of Concept:**

Add to `barretenberg/cpp/src/barretenberg/stdlib/primitives/memory/rom_table.test.cpp`:

```
TEST(RomTableMoveBug, MovedFromAccessCausesInconsistentState)
{
    using Builder = UltraCircuitBuilder;
    using field_ct = stdlib::field_t<Builder>;
    using witness_ct = stdlib::witness_t<Builder>;
    using rom_table_ct = stdlib::rom_table<Builder>;

    Builder builder;

    std::vector<field_ct> values;
    for (size_t i = 0; i < 5; ++i) {
        values.emplace_back(witness_ct(&builder, bb::fr(i)));
    }
    rom_table_ct table1(values);

    // Force initialization
    [[maybe_unused]] auto init = table1[field_ct(witness_ct(&builder, 0))];

    rom_table_ct table2 = std::move(table1);

    // table2 works
    [[maybe_unused]] auto y = table2[0];

    // Uncomment to trigger segfault:
    // [[maybe_unused]] auto x = table1[0];

    EXPECT_EQ(table1.size(), 5);  // BUG: should be 0 after move
}
```

Run:

```
cmake --build --preset default --target stdlib_primitives_tests
./build/bin/stdlib_primitives_tests --gtest_filter="*RomTableMoveBug*"
```

Output (crash line commented):

```
[==========] Running 1 test from 1 test suite.
[----------] 1 test from RomTableMoveBug
```

```
[ RUN      ] RomTableMoveBug.MovedFromAccessCausesInconsistentState
[       OK ] RomTableMoveBug.MovedFromAccessCausesInconsistentState (0 ms)
[  PASSED  ] 1 test.
```

Output (crash line uncommented):

```
[==========] Running 1 test from 1 test suite.
[----------] 1 test from RomTableMoveBug
[ RUN      ] RomTableMoveBug.MovedFromAccessCausesInconsistentState
Segmentation fault (core dumped)
```

## Tool Used

Manual Review

## Recommendation

Implement custom move constructors that reset `length` to zero after moving.

# Issue L-2: Silent fallback to zero in get_curve_b() would enforce wrong curve equation for unsupported field types [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/49

This issue has been acknowledged by the team but won't be fixed at this time.

## Summary

`get_curve_b()` silently returns 0 for unsupported field types instead of failing, which would cause elliptic curve constraints to enforce the wrong curve equation. This is currently not triggered since Aztec only uses BN254 and Grumpkin curves, both of which are handled correctly.

## Vulnerability Detail

The function returns the curve parameter `b` for BN254 and Grumpkin, but defaults to 0 for other field types. This causes the doubling constraint to enforce $y^2 = x^3$ instead of $y^2 = x^3 + b$. The same pattern exists in `ultra_circuit_builder.cpp:1979-1986`.

## Impact

Latent soundness violation. If new curves are added without updating this function, proofs with invalid elliptic curve operations could verify.

## Code Snippet

```
static constexpr FF get_curve_b()
{
if constexpr (FF::modulus == bb::fq::modulus) {
    return bb::g1::curve_b;
} else if constexpr (FF::modulus == grumpkin::fq::modulus) {
    return grumpkin::g1::curve_b;
} else {
    return 0;  // Silent wrong value
}
}
```

barretenberg/relations/elliptic_relation.hpp#L30

## Tool Used

Manual Review

## Recommendation

Replace the silent fallback with a compile-time error and optionally centralize the curve parameter logic.

Example:

```
static constexpr FF get_curve_b()
{
    static_assert(FF::modulus == bb::fq::modulus ||
                  FF::modulus == grumpkin::fq::modulus,
                  "EllipticRelation: unsupported curve");

    if constexpr (FF::modulus == bb::fq::modulus) {
        return bb::g1::curve_b;
    } else {
        return grumpkin::g1::curve_b;
    }
}
```

# Issue L-3: Missing TBB linkage causes build failure when parallel algorithms are enabled [RESOLVED]

Source: https://github.com/sherlock-audit/2026-01-aztec-network-jan-14th/issues/51

## Summary

When `ENABLE_PAR_ALGOS=ON` is set and TBB is found, the CMake configuration enables parallel algorithm support but fails to link the TBB library, causing linker errors.

## Vulnerability Detail

In `cmake/threading.cmake`, when TBB is found, the code only prints a status message but never links the library:

```
if(ENABLE_PAR_ALGOS)
    find_package(TBB QUIET OPTIONAL_COMPONENTS tbb)
    if(${TBB_FOUND})
        message(STATUS "std::execution parallel algorithms are enabled.")
    else()
        message(STATUS "Could not locate Intel TBB, disabling std::execution
        ↪  parallel algorithms.")
        add_definitions(-DNO_PAR_ALGOS)
    endif()
else()
    message(STATUS "std::execution parallel algorithms are disabled.")
    add_definitions(-DNO_PAR_ALGOS)
endif()
```

When building with `-DENABLE_PAR_ALGOS=ON`, the code compiles but fails at link time with undefined references to TBB symbols:

```
.
.
undefined reference to
↪  `tbb::detail::r1::deallocate(tbb::detail::d1::small_object_pool&, void*,
↪  unsigned long, tbb::detail::d1::execution_data const&)'
undefined reference to `tbb::detail::r1::notify_waiters(unsigned long)'
undefined reference to
↪  `tbb::detail::r1::allocate(tbb::detail::d1::small_object_pool*&, unsigned long,
↪  tbb::detail::d1::execution_data const&)'
undefined reference to `tbb::detail::r1::spawn(tbb::detail::d1::task&,
↪  tbb::detail::d1::task_group_context&)'
.
.
```

## Impact

The `ENABLE_PAR_ALGOS` build option is broken.

## Code Snippet

```
if(ENABLE_PAR_ALGOS)
    find_package(TBB QUIET OPTIONAL_COMPONENTS tbb)
    if(${TBB_FOUND})
        message(STATUS "std::execution parallel algorithms are enabled.")
    else()
        message(STATUS "Could not locate Intel TBB, disabling std::execution
        ↪  parallel algorithms.")
        add_definitions(-DNO_PAR_ALGOS)
    endif()
```

barretenberg/cpp/cmake/threading.cmake#L29

## Tool Used

Manual Review

## Recommendation

Add `link_libraries(TBB::tbb)` after TBB is found:

```
if(ENABLE_PAR_ALGOS)
    find_package(TBB QUIET OPTIONAL_COMPONENTS tbb)
    if(${TBB_FOUND})
        message(STATUS "std::execution parallel algorithms are enabled.")
        link_libraries(TBB::tbb)
    else()
        message(STATUS "Could not locate Intel TBB, disabling std::execution
        ↪  parallel algorithms.")
        add_definitions(-DNO_PAR_ALGOS)
    endif()
```

After applying this fix, the build succeeds with parallel algorithms enabled.

# Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.